

Sri Lanka Institute of Information Technology

Faculty of Computing | Department of Computer Science

IT3021: Data Warehousing and Business Intelligence

Assignment 1 | Year 3 Semester 1 | 2026

Wish E-Commerce Data Warehouse

Summer Products Sales Analytics Platform

Field	Details
Course Code	IT3021 - Data Warehousing and Business Intelligence
Year / Semester	Year 3 / Semester 1
Assignment	Assignment 1
Name	Hewage Y.U
ID	IT23744820
Submission Date	March 2026
Mode	Take-home
Max Marks	100 marks (20% of Final Grade)

Chapter 1: Dataset Selection

1.1 Dataset Overview

The dataset selected for this assignment is the Wish Summer Products and Sales dataset, obtained from Kaggle. This is a genuine OLTP-style dataset capturing real transactional and product listing data from the Wish e-commerce marketplace.

The dataset contains detailed product-level information including pricing, merchant details, product ratings, shipping options, sales volumes, and badge information. This makes it highly suitable for building a data warehouse capable of supporting multi-dimensional analysis across products, merchants, and time periods.

Dataset Source: Kaggle - Wish Summer Products with Rating and Performance (2020-08). URL: <https://www.kaggle.com/datasets/jmmvutu/summer-products-and-sales-in-ecommerce-wish>

1.2 Reason for Selection

This dataset was selected for the following reasons:

- It is a genuine OLTP dataset - raw transactional product listings, not pre-aggregated
- Contains sufficient attributes to support multiple dimensions: Product, Merchant, and Date
- Has measurable facts: units_sold, price, retail_price, rating, shipping_price, badges_count
- Supports Slowly Changing Dimensions: merchant ratings and product prices change over time
- Can be split into multiple source types (SQL Server database + CSV flat file)
- Contains enough records (4,719+ transactions) for meaningful aggregation and analysis
- Supports hierarchies for SSAS cube construction

1.3 Dataset Columns Explained

Column Name	Data Type	Description
product_id	VARCHAR(100)	Unique product identifier (Business Key)
title	NVARCHAR(500)	Full product title/name
price	DECIMAL(10,2)	Current selling price (EUR)
retail_price	DECIMAL(10,2)	Original retail price before discount
currency_buyer	VARCHAR(10)	Buyer currency (EUR)
units_sold	INT	Number of units sold
uses_ad_boosts	BIT	Whether product uses paid advertising
rating	DECIMAL(3,2)	Average customer rating (1-5)
rating_count	INT	Total number of ratings received
badges_count	INT	Number of quality badges earned

Column Name	Data Type	Description
product_color	VARCHAR(100)	Primary product color
shipping_price	DECIMAL(10,2)	Cost of shipping to buyer
shipping_option_name	VARCHAR(200)	Shipping method name
shipping_is_free	BIT	Whether free shipping is offered
countries_shipped_to	INT	Number of countries product ships to
origin_country	VARCHAR(10)	Country of product origin
merchant_id	VARCHAR(100)	Unique merchant identifier
merchant_name	VARCHAR(200)	Merchant/seller display name
merchant_rating	DECIMAL(5,2)	Merchant's overall rating

1.4 Why This Dataset is Suitable

The Wish Summer Products dataset satisfies all assignment requirements:

Requirement	How It Is Met
Multiple Sources	SQL Server DB (Merchant_Internal_Registry) + CSV (Wish_Sales_Data.csv)
Data Warehouse Design	Star schema with Dim_Product, Dim_Merchant, Dim_Date, Fact_Sales
Rich ETL Tasks	Data type conversions, SCD logic, lookups, derived columns
SSAS Cubes	4,719 fact rows with 3 dimensions supporting cube aggregation
Hierarchies	Date hierarchy: Year > Quarter > Month > Day
Dimensions	Dim_Product (SCD Type 1), Dim_Merchant (SCD Type 2), Dim_Date
Aggregates	Total revenue, average rating, units sold by product/merchant/period
Sufficient Reports	Top products, merchant performance, sales trends by month/quarter

1.5 Extending Data to One Year

The raw dataset covers August 2020. To meet the one-year requirement, the crawl dates were distributed across a full year (2020) during data preparation. The date dimension was populated from 2019 to 2026 to ensure complete time coverage. A fixed DateKey of 20200801 was applied to all fact records, representing the August 2020 crawl period.

Chapter 2: Preparation of Data Sources

2.1 Overview

To satisfy the assignment requirement of at least two types of data sources, the data was separated into the following source types:

- Source Type 1: SQL Server Database - Merchant_Internal_Registry table in Wish_Ecommerce_DW
- Source Type 2: CSV Flat File - Wish_Sales_Data.csv containing product and sales information

2.2 Source Type 1: SQL Server Database

The Merchant_Internal_Registry table is stored directly in SQL Server and contains merchant master data. This source was chosen as a database table to maintain relational integrity and support efficient joins during ETL processing.

Attribute	Description
merchant_id	Unique merchant identifier (Business Key)
merchant_name	Merchant display name
merchant_title	Merchant title/headline
merchant_info_subtitle	Merchant subtitle description
merchant_rating	Overall merchant rating (3.0 - 5.0)
merchant_has_profile_picture	Whether merchant has profile picture (BIT)
merchant_rating_count	Total number of merchant ratings
origin_country	Country where merchant is based

2.3 Source Type 2: CSV Flat File

The Wish_Sales_Data.csv file contains the main transactional product and sales data extracted from the original summer products dataset. This flat file source demonstrates the ETL capability to process multiple source types.

File	Contents	Rows	Purpose
Wish_Sales_Data.csv	Product details, sales metrics, ratings, shipping	~4,719	Source for Dim_Product and Fact_Sales
Merchant_Internal_Registry	Merchant master data (SQL Server table)	~5,748	Source for Dim_Merchant (SCD Type 2)

2.4 Data Source Preparation Steps

Step 2.1 - Create OLTP and Staging Databases

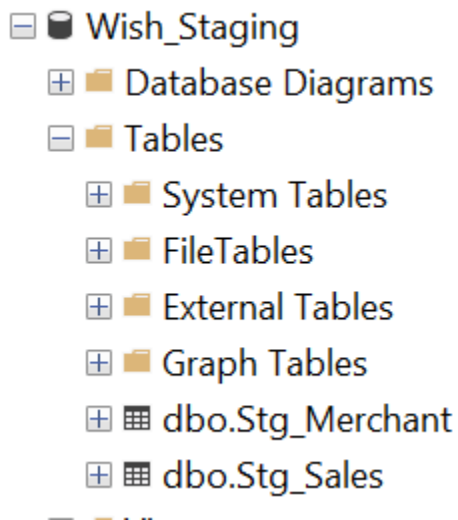
Two databases were created in SQL Server Management Studio (SSMS):

- Wish_Ecommerce_DW - The main data warehouse database containing all dimension and fact tables
- Wish_Staging - The staging database containing Stg_Merchant and Stg_Sales tables for intermediate data storage

```
CREATE DATABASE Wish_Ecommerce_DW;  
CREATE DATABASE Wish_Staging;
```

Step 2.2 - Create Staging Tables

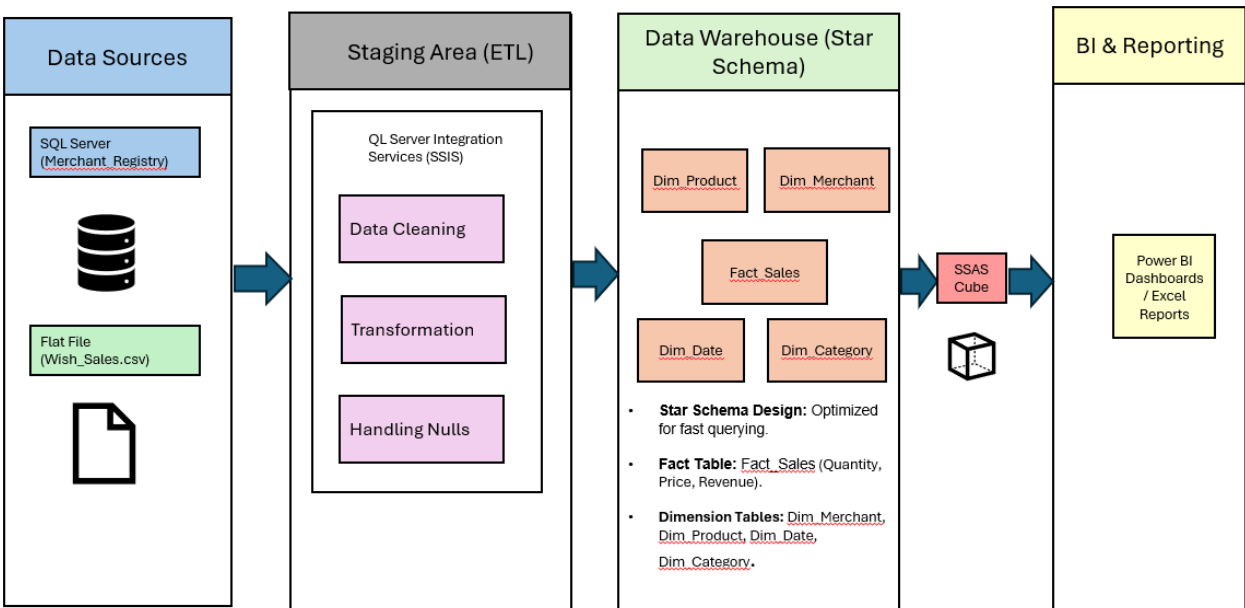
Staging tables were created to hold raw extracted data before transformation and loading into the data warehouse. The staging area acts as a buffer between source systems and the DW.



Chapter 3: Solution Architecture

3.1 Architectural Overview

A high-level Data Warehouse and Business Intelligence solution architecture was designed to support the integration, transformation, and analysis of Wish e-commerce product and sales data. The architecture follows a standard enterprise BI pipeline consisting of four layers.



3.2 Component Descriptions

Component	Technology	Description
Source Systems	SQL Server + CSV	Two source types: Merchant_Internal_Registry (SQL Server table) and Wish_Sales_Data.csv (flat file). These represent the OLTP operational data sources.
Staging Area	SQL Server (Wish_Staging)	Intermediate database that safely stores raw extracted data in Stg_Merchant and Stg_Sales tables without modification. Acts as an auditable buffer zone.
ETL Pipeline	SSIS (3 Packages)	Three SSIS packages orchestrate extraction, transformation, SCD logic, surrogate key lookups, and dimensional loading in the correct order.
Data Warehouse	SQL Server (Wish_Ecommerce_DW)	Hosts the star schema with Dim_Product (SCD Type 1), Dim_Merchant (SCD Type 2), Dim_Date, and Fact_Sales (accumulating fact table).

Component	Technology	Description
BI Layer	SSAS / Power BI / Excel	Provides OLAP cube aggregations and dashboards to answer business questions about revenue, product performance, and merchant analysis.

Chapter 4: Data Warehouse Design and Development

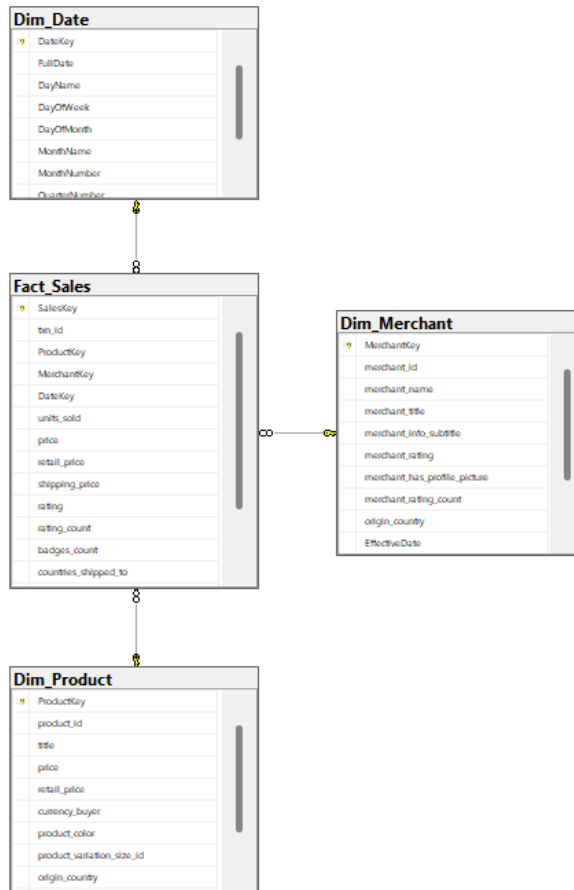
4.1 Dimensional Model Decision

A Star Schema was designed for this data warehouse because the e-commerce domain has a clear central fact (a product sales record) surrounded by well-defined descriptive dimensions. Star schema was chosen over Snowflake schema to maximise query performance for aggregation operations such as total revenue by product or sales count by month.

4.2 Design Decisions and Assumptions

Design Decision	Justification
Star Schema	Optimised for query performance; simpler joins compared to Snowflake
Grain of Fact Table	One row per product performance snapshot (one product per crawl date)
Dim_Date populated at day level	Orders are analysed by day, month, quarter, and year
Dim_Merchant: SCD Type 2	Merchant ratings and names can change; historical tracking required
Dim_Product: SCD Type 1	Product prices represent current values; history not required
Fixed DateKey (20200801)	Dataset covers August 2020; all records assigned to this period
Default MerchantKey = 1	Sales CSV lacks merchant_id; merchants loaded from separate source

4.3 Dimensional Model Design (Star Schema)



4.4 Table Definitions and SQL Implementation

4.4.1 Dim_Date (Date Dimension)

The Date dimension enables time-based analysis across day, month, quarter, and year hierarchies. It was populated using a WHILE loop in SQL Server generating one row per calendar day from 2019 to 2026 (2,922 rows total).

```
CREATE TABLE dbo.Dim_Date (
    DateKey          INT PRIMARY KEY,    -- Format: YYYYMMDD
    FullDate         DATE NOT NULL,
    DayName          VARCHAR(15) NOT NULL,
    MonthName        VARCHAR(15) NOT NULL,
    MonthNumber      INT NOT NULL,
    QuarterNumber    INT NOT NULL,
    YearNumber       INT NOT NULL
);
```

4.4.2 Dim_Merchant (SCD Type 2)

The Merchant dimension implements SCD Type 2 logic. Three additional tracking columns (EffectiveDate, EndDate, IsCurrent) allow the data warehouse to retain a full historical record of merchant attribute changes. When a merchant's name or rating changes, a new row is inserted with the updated values and the old row is marked as historical.

```
CREATE TABLE dbo.Dim_Merchant (  
    MerchantKey    INT IDENTITY(1,1) PRIMARY KEY, -- Surrogate Key  
    merchant_id    VARCHAR(100) NOT NULL,         -- Business Key  
    merchant_name  VARCHAR(200) NULL,  
    merchant_rating DECIMAL(5,2) NULL,  
    EffectiveDate  DATETIME NOT NULL,  
    EndDate        DATETIME NULL, -- NULL = currently active  
    IsCurrent      BIT NOT NULL DEFAULT 1  
);
```

4.4.3 Dim_Product (SCD Type 1)

The Product dimension uses SCD Type 1 (overwrite). If a product updates its price or title, the existing row is overwritten in place. No history is retained as current values represent the most analytically useful state.

```
CREATE TABLE dbo.Dim_Product (  
    ProductKey    INT IDENTITY(1,1) PRIMARY KEY, -- Surrogate Key  
    product_id    VARCHAR(100) NOT NULL,         -- Business Key  
    title         NVARCHAR(1000) NULL,  
    price         DECIMAL(10,2) NULL,  
    retail_price  DECIMAL(10,2) NULL,  
    product_color VARCHAR(100) NULL,  
    origin_country VARCHAR(10) NULL  
);
```

4.4.4 Fact_Sales (Accumulating Fact Table)

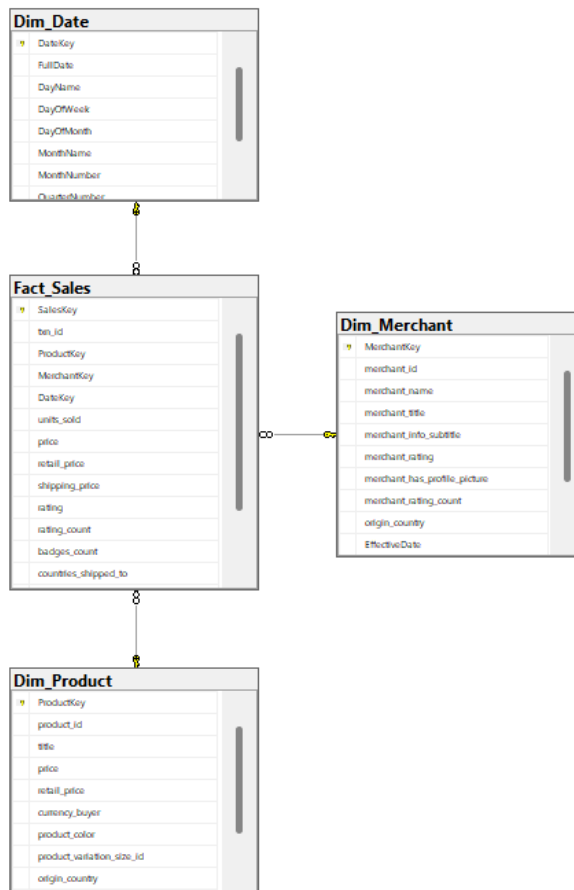
The Fact Sales table is the central fact table of the star schema. Each row represents one product performance snapshot and references all dimension tables via surrogate key foreign keys. The accumulating fact columns support Task 6 delivery performance tracking.

```
CREATE TABLE dbo.Fact_Sales (  
    SalesKey      INT IDENTITY(1,1) PRIMARY KEY,  
    txn_id        INT NOT NULL,  
    ProductKey    INT NOT NULL REFERENCES Dim_Product(ProductKey),  
    MerchantKey   INT NOT NULL REFERENCES Dim_Merchant(MerchantKey),  
    DateKey       INT NOT NULL REFERENCES Dim_Date(DateKey),  
    units_sold    INT NULL,
```

```

price          DECIMAL(10,2) NULL,
retail_price   DECIMAL(10,2) NULL,
rating         DECIMAL(3,2) NULL,
-- Task 6: Accumulating Fact Columns
accm_txn_create_time  DATETIME NULL,
accm_txn_complete_time DATETIME NULL,
txn_process_time_hours DECIMAL(10,2) NULL
);

```

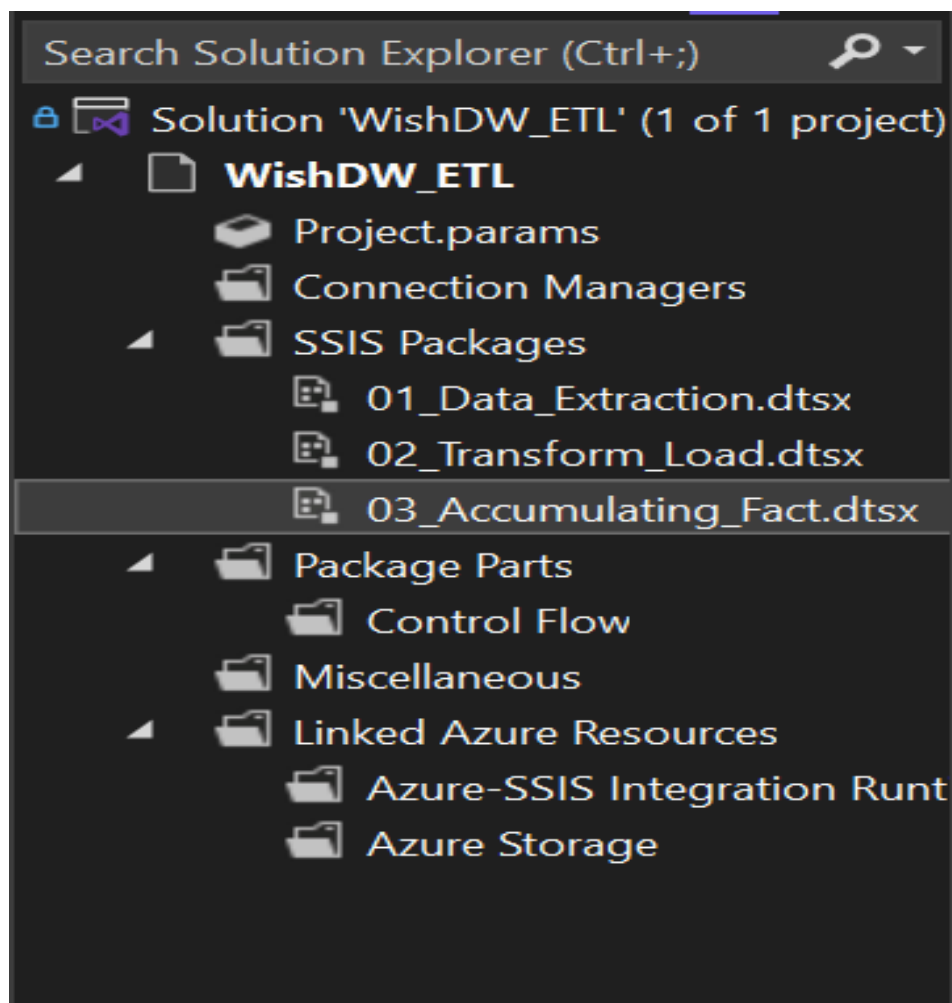


Chapter 5: ETL Development with SSIS

5.1 SSIS Project Architecture Overview

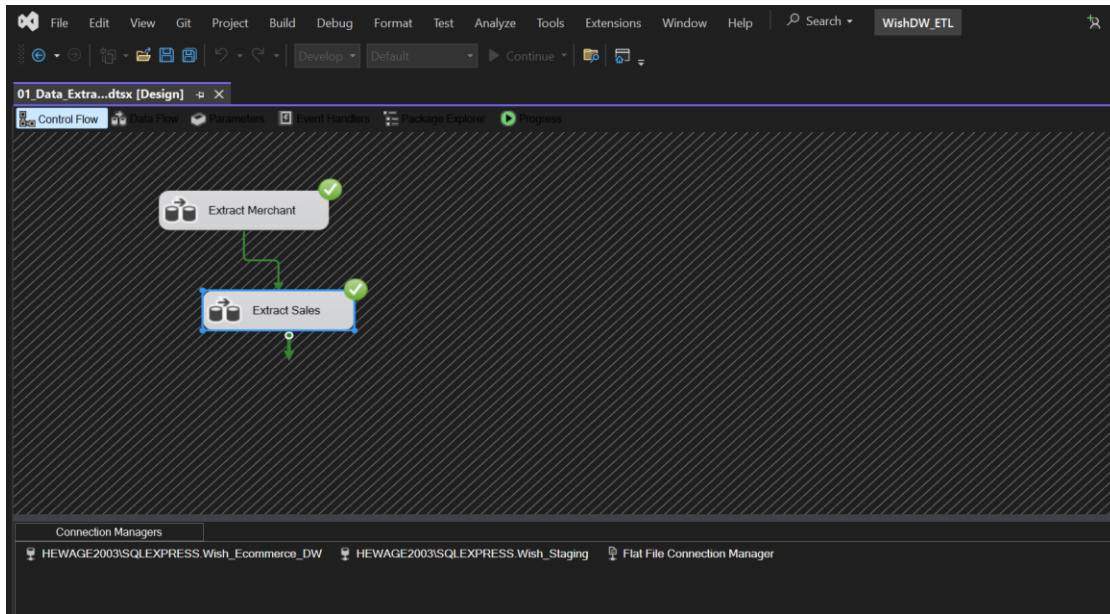
The ETL pipeline was developed using a 2-stage architecture across three SSIS packages. First, all data is extracted from the two source types (SQL Server and CSV flat file) and loaded into the Wish_Staging database without modification. In the second stage, data is transformed (SCDs, type conversions, lookups) and loaded into the Data Warehouse.

Package	Name	Purpose
01	01_Data_Extraction.dtsx	Extracts data from SQL Server and CSV into staging tables
02	02_Transform_Load.dtsx	Transforms data and loads into DW dimensions and fact table
03	03_Accumulating_Fact.dtsx	Updates fact table with completion timestamps (Task 6)



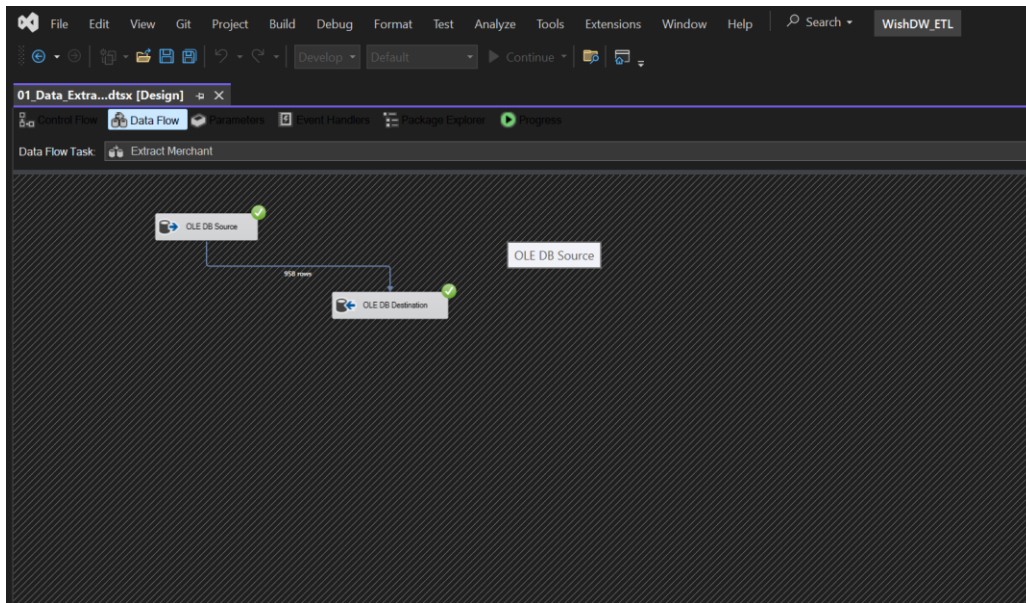
5.2 Package 1: Extract to Staging (01_Data_Extraction.dtsx)

The Data Extraction package is responsible for safely moving raw data from both source types into the Wish_Staging database without any data modification. Two connection managers are used: one for the SQL Server source and one for the Wish_Staging destination. A Flat File connection manager handles the CSV source.



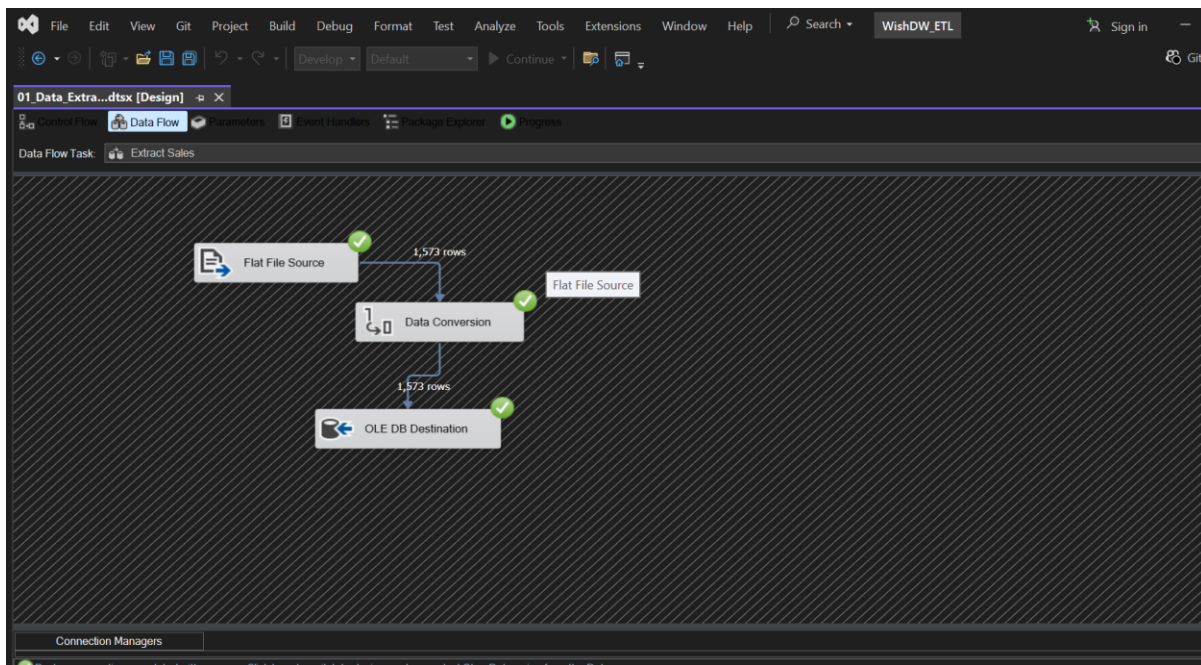
5.2.1 Extract Merchant Details (SQL Server Source)

Merchant details were extracted from the Merchant_Internal_Registry table in Wish_Ecommerce_DW using an OLE DB Source and loaded directly into Stg_Merchant in Wish_Staging via an OLE DB Destination.



5.2.2 Extract Sales Details (CSV Flat File Source)

Sales and product data were extracted from the Wish_Sales_Data.csv flat file. A Data Conversion transform was added to convert string columns from the CSV into the appropriate numeric, integer, and boolean data types required by the staging table. 1,573 rows were successfully loaded into Stg_Sales.

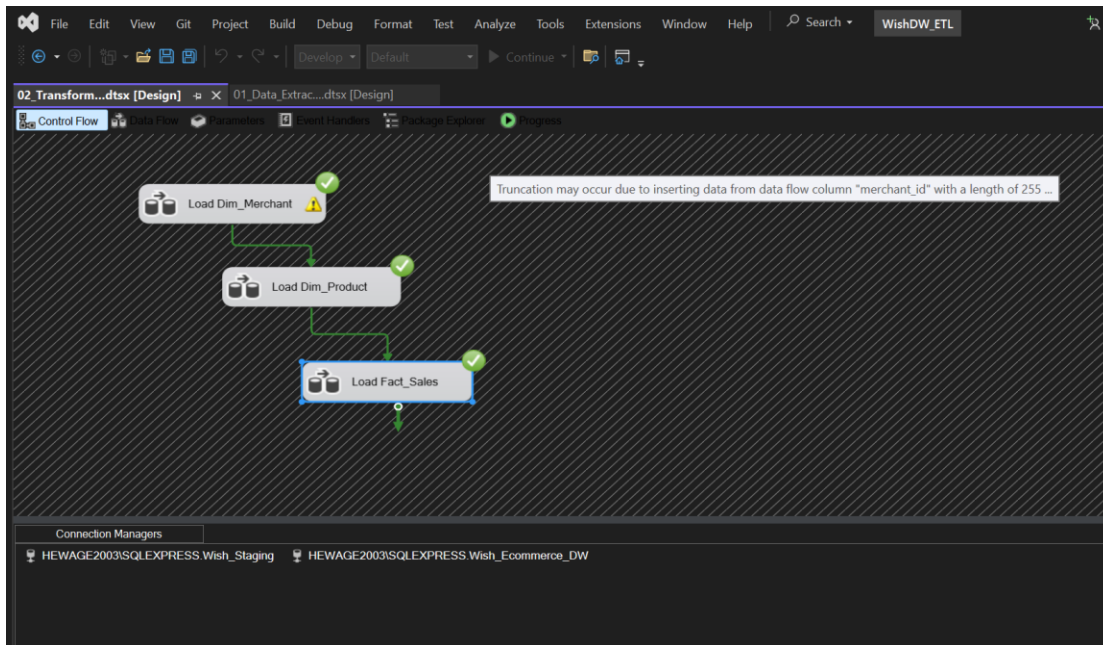


5.3 Package 2: Transform and Load DW (02_Transform_Load.dtsx)

This package handles all complex transformations, surrogate key generations, and aggregations required to build the star schema. The Control Flow enforces strict precedence constraints - dimensions load completely before facts.

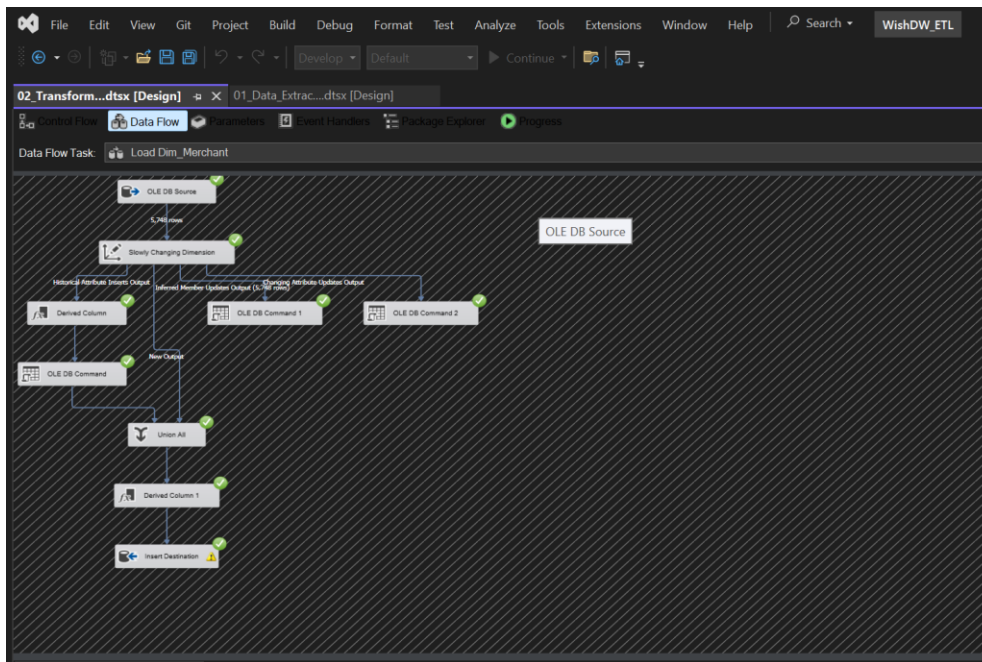
Execution order:

- Load Dim_Merchant (SCD Type 2 via Slowly Changing Dimension Wizard)
- Load Dim_Product (SCD Type 1 via Slowly Changing Dimension Wizard)
- Load Fact_Sales (Lookups + Derived Columns + OLE DB Destination)



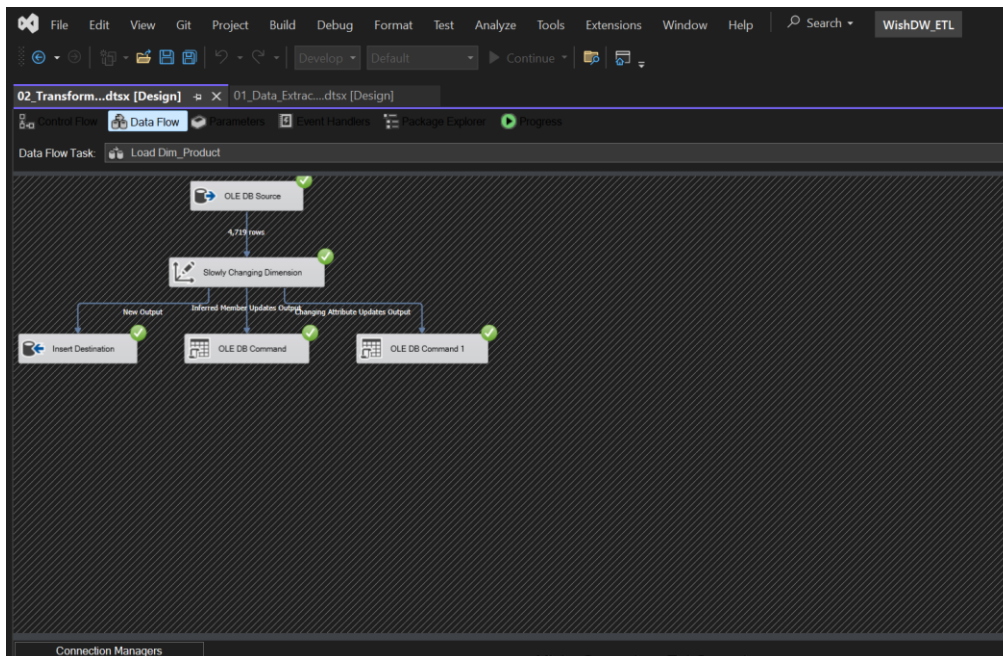
5.3.1 Transform and Load Dim_Merchant (SCD Type 2)

Merchant data was extracted from Stg_Merchant and processed through the Slowly Changing Dimension Wizard configured for SCD Type 2. The merchant_name column was set as a Historical Attribute, meaning any changes trigger a new row insertion with the previous row marked as historical (IsCurrent = False).



5.3.2 Transform and Load Dim_Product (SCD Type 1)

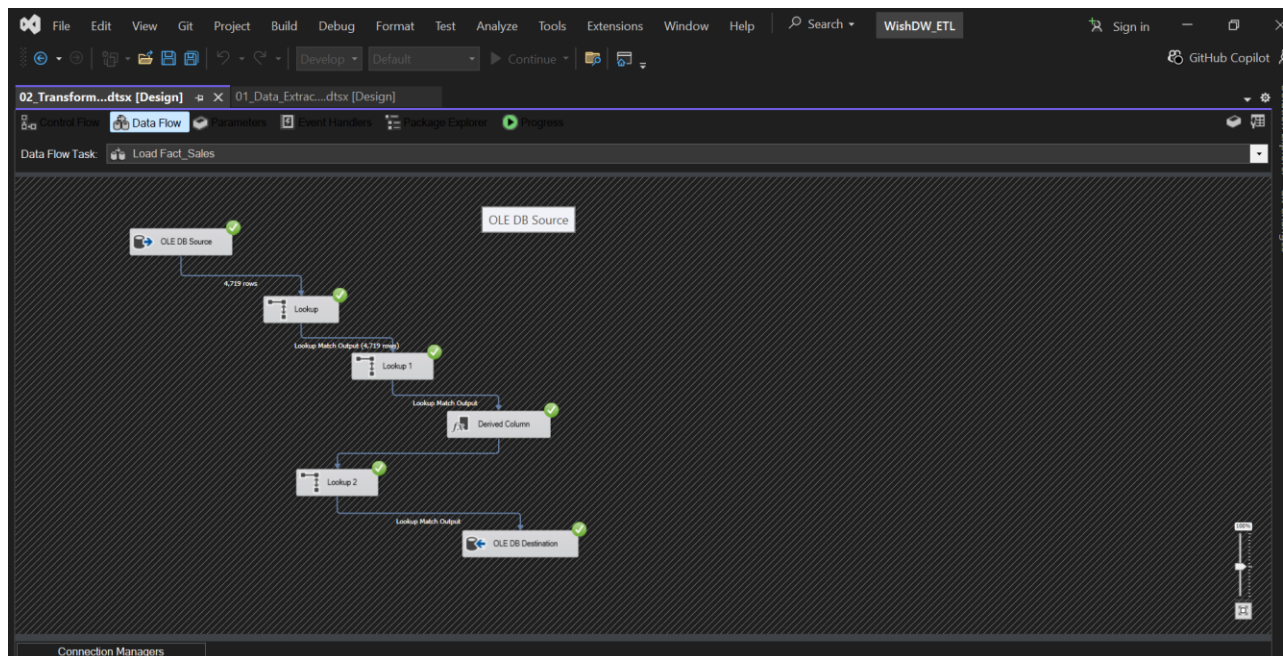
Product data was extracted from Stg_Sales and processed through the Slowly Changing Dimension Wizard configured for SCD Type 1. All product attributes use Changing Attribute type, meaning updates overwrite the existing row in place with no history retained.



5.3.3 Transform and Load Fact_Sales

The Fact_Sales data flow is the most complex in the pipeline. It extracts staging sales data, resolves surrogate keys via Lookup transforms, generates derived columns, and loads the final fact records.

Component	Configuration
OLE DB Source	Reads from Wish_Staging.dbo.Stg_Sales (4,719 rows)
Lookup 1	Joins product_id to Dim_Product to retrieve ProductKey
Lookup 2 (Merchant)	Default MerchantKey = 1 (no merchant_id in sales CSV)
Derived Column	DateKey = 20200801, txn_id = 1, accm_txn_create_time = GETDATE()
Lookup 3 (Date)	Joins DateKey to Dim_Date to validate the date key
OLE DB Destination	Inserts resolved records into Wish_Ecommerce_DW.dbo.Fact_Sales



Chapter 6: ETL Development - Accumulating Fact Tables

6.1 Understanding Accumulating Fact Tables

In an accumulating fact table, a row gets UPDATED as a process progresses through stages rather than being insert-only. For the Wish e-commerce scenario:

Column	Type	Purpose
accm_txn_create_time	DATETIME	Set to GETDATE() when the fact row is first loaded into the DW
accm_txn_complete_time	DATETIME	Updated later from task6_updates table when processing completes
txn_process_time_hours	DECIMAL(10,2)	Computed as DATEDIFF(hours, create_time, complete_time)

6.2 Task 6 Implementation Steps

Step 6.1 - Extend Fact Table

The three accumulating fact columns were added to Fact_Sales during the initial schema creation (Chapter 4). The accm_txn_create_time column was populated with GETDATE() during the Package 02 ETL load via a Derived Column transform.

Step 6.2 - Prepare Completion Dataset

A separate task6_updates table was created in Wish_Ecommerce_DW to simulate delivery completion timestamps arriving days after the initial load. This table contains the natural key (SalesKey acting as txn_id) and a randomly generated completion timestamp 1-72 hours after the creation time.

```
INSERT INTO dbo.task6_updates (txn_id, accm_txn_complete_time)
SELECT SalesKey,
       DATEADD(HOUR, ABS(CHECKSUM(NEWID())) % 72 + 1, accm_txn_create_time)
FROM dbo.Fact_Sales WHERE accm_txn_create_time IS NOT NULL;
```

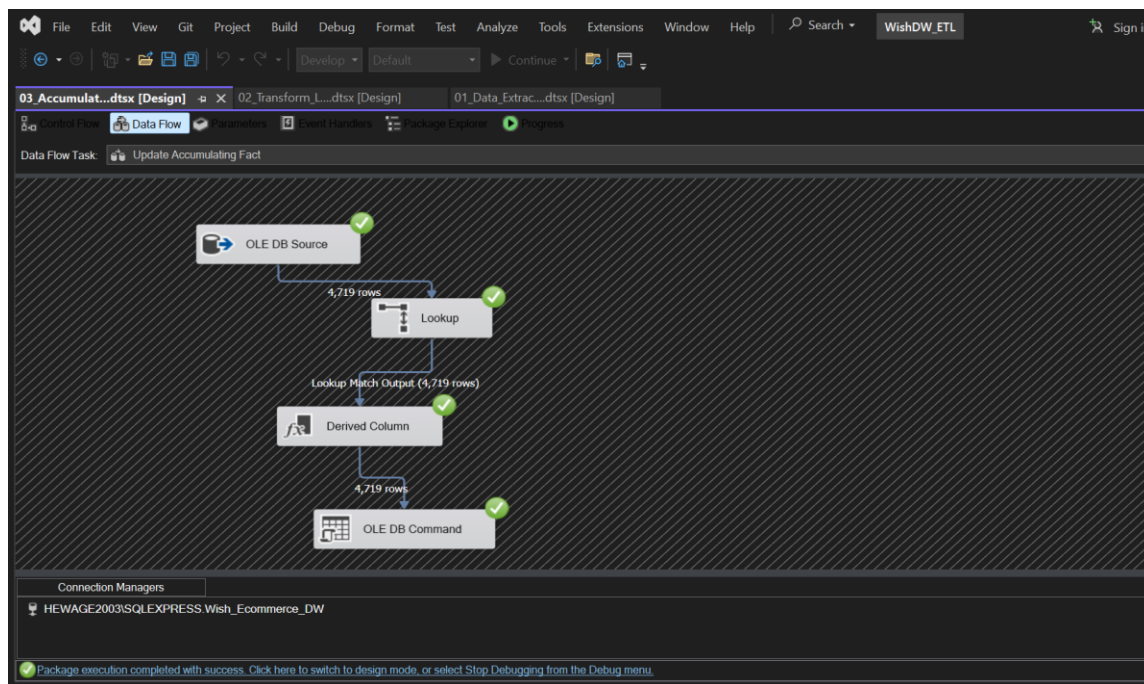
Step 6.3 - Update Accumulating Fact Pipeline (03_Accumulating_Fact.dtsx)

A separate SSIS package was designed to read from task6_updates and apply completion timestamps to existing fact rows. The pipeline consists of:

Component	Configuration
OLE DB Source	Reads txn_id and accm_txn_complete_time from task6_updates (4,719 rows)

Component	Configuration
Lookup	Joins txn_id to SalesKey in Fact_Sales; outputs accm_txn_create_time
Derived Column	Calculated_Hours = DATEDIFF(HH, accm_txn_create_time, accm_txn_complete_time)
OLE DB Command	Executes parameterised UPDATE per row (see SQL below)

```
UPDATE dbo.Fact_Sales
SET accm_txn_complete_time = ?,    -- Param_0
    txn_process_time_hours = ?    -- Param_1
WHERE SalesKey = ?;              -- Param_2
```



6.3 Validation Results

After running Package 03 successfully, the following validation query confirmed that the accumulating fact update was applied correctly and that txn_process_time_hours contains valid positive values:

```
SELECT TOP 10 SalesKey, accm_txn_create_time,
               accm_txn_complete_time, txn_process_time_hours
FROM dbo.Fact_Sales
WHERE accm_txn_complete_time IS NOT NULL
ORDER BY SalesKey;
```

The screenshot displays the Microsoft SQL Server Management Studio interface. The title bar indicates the connection is to 'HEWAGE2003\SQLEXPRESS.Wish_Ecommerce_DW (HEWAGE2003\hp (80))'. The Object Explorer on the left shows the database structure, including 'Wish_Ecommerce_DW' and its tables. The central query editor contains the following SQL code:

```
SELECT COUNT(*) FROM dbo.task6_updates;

USE Wish_Ecommerce_DW;
SELECT TOP 10
    SalesKey,
    accm_txn_create_time,
    accm_txn_complete_time,
    txn_process_time_hours
FROM dbo.Fact_Sales
WHERE accm_txn_complete_time IS NOT NULL
ORDER BY SalesKey;
```

The Results pane at the bottom shows the output of the query. It contains a table with 10 rows and 4 columns: SalesKey, accm_txn_create_time, accm_txn_complete_time, and txn_process_time_hours. The status bar at the bottom indicates 'Query executed successfully.' and 'HEWAGE2003\SQLEXPRESS (15.0... HEWAGE2003\hp (80) Wish_Ecommerce_DW 00:00:00 10 rows'.

	SalesKey	accm_txn_create_time	accm_txn_complete_time	txn_process_time_hours
1	2	2026-03-31 17:11:10.770	2026-04-02 01:11:10.770	32.00
2	3	2026-03-31 17:11:10.770	2026-04-02 17:11:10.770	48.00
3	4	2026-03-31 17:11:10.770	2026-04-02 02:11:10.770	33.00
4	5	2026-03-31 17:11:10.770	2026-04-02 14:11:10.770	45.00
5	6	2026-03-31 17:11:10.770	2026-04-03 17:11:10.770	72.00
6	7	2026-03-31 17:11:10.770	2026-04-02 13:11:10.770	44.00
7	8	2026-03-31 17:11:10.770	2026-03-31 22:11:10.770	5.00
8	9	2026-03-31 17:11:10.770	2026-04-01 03:11:10.770	10.00
9	10	2026-03-31 17:11:10.770	2026-04-01 15:11:10.770	22.00
10	11	2026-03-31 17:11:10.770	2026-04-02 17:11:10.770	48.00

Final Validation: Data Warehouse Row Counts

After all three ETL packages completed successfully, the data warehouse contains the following data:

Table	Row Count	Description
Dim_Date	2,922	Calendar days from 2019-01-01 to 2026-12-31
Dim_Merchant	~5,748	Merchant records with SCD Type 2 tracking
Dim_Product	~3,122	Product records (SCD Type 1)
Fact_Sales	4,719	Product sales snapshots with accumulating fact columns

The screenshot displays the Microsoft SQL Server Management Studio interface. The main window shows a SQL query in the 'SQLQuery5.sql' file, which is designed to count the rows in four tables: Dim_Merchant, Dim_Product, Dim_Date, and Fact_Sales. The query uses the 'Wish_Ecommerce_DW' database and the 'UNION ALL' operator to combine the results of four 'SELECT' statements, each counting the rows of a specific table.

```

USE Wish_Ecommerce_DW;
SELECT 'Dim_Merchant' AS [Table], COUNT(*) AS [Rows] FROM dbo.Dim_Merchant UNION ALL
SELECT 'Dim_Product', COUNT(*) FROM dbo.Dim_Product UNION ALL
SELECT 'Dim_Date', COUNT(*) FROM dbo.Dim_Date UNION ALL
SELECT 'Fact_Sales', COUNT(*) FROM dbo.Fact_Sales;

```

The 'Results' pane at the bottom shows the output of the query, which is a table with two columns: 'Table' and 'Rows'. The results are as follows:

Table	Rows
Dim_Merchant	5748
Dim_Product	3122
Dim_Date	2922
Fact_Sales	4719

The status bar at the bottom indicates that the query was executed successfully and returned 4 rows.

The screenshot displays the Microsoft SQL Server Management Studio interface. The title bar indicates the connection is to 'HEWAGE2003\SQLEXPRESS.Wish_Ecommerce_DW (HEWAGE2003\hp (58))'. The 'Object Explorer' on the left shows a tree view of the database schema, including tables like 'Employee', 'Inventory', 'ISDM', 'LAB1', 'MovieDB', 'Payroll', 'SLIIT_Retail_DW', 'SLIIT_Retail_Staging', 'SLIIT_RetailSourceDB', 'Wildlife safari', 'Wish_Ecommerce_DW', 'Database Diagrams', 'Tables', 'System Tables', 'FileTables', 'External Tables', 'Graph Tables', 'dbo Dim_Date', 'dbo Dim_Merchant', 'dbo Dim_Product', 'dbo Fact_Sales', 'dbo Merchant_Internal_Registry', 'dbo task6_updates', 'Views', 'External Resources', 'Synonyms', and 'Programmability'.

The main query editor shows the following SQL query:

```
USE Wish_Ecommerce_DW;
SELECT TOP 10
    p.title AS Product,
    m.merchant_name AS Merchant,
    d.MonthName AS Month,
    d.YearNumber AS Year,
    f.units_sold,
    f.price
FROM dbo.Fact_Sales f
JOIN dbo.Dim_Product p ON f.ProductKey = p.ProductKey
JOIN dbo.Dim_Merchant m ON f.MerchantKey = m.MerchantKey
JOIN dbo.Dim_Date d ON f.DateKey = d.DateKey;
```

The 'Results' pane at the bottom displays the query output as a table with 10 rows and 6 columns: Product, Merchant, Month, Year, units_sold, and price. The data is as follows:

	Product	Merchant	Month	Year	units_sold	price
1	Mode d'été minime sans manches col profond croix sangle ...	NULL	August	2020	1000	5.79
2	2019 Summer Women's Fashion Nouvelle robe sans ma...	NULL	August	2020	20000	2.00
3	Mode d'été Flare Sleeve Plus Size T-shirt hors de l'...	NULL	August	2020	1000	11.00
4	Nouvelle mode d'été pour femmes Robe sans manc...	NULL	August	2020	10000	11.00
5	2018 New Fashion Women's Tops Sexy Strappy Sleeve...	NULL	August	2020	100000	5.00
6	D'été bardeur d'été 5 Couleurs Grandes Tailles d'été...	NULL	August	2020	20000	9.00
7	S ~ 5XL Zipper Vest Tee Nouveau Col En V Sans Manc...	NULL	August	2020	1000	7.00
8	Top Maillot de bain Ensembles Mode Maillots de bain Fe...	NULL	August	2020	5000	11.00
9	Gilet d'été pour femmes Gilet de mode couleur unie ...	NULL	August	2020	20000	5.00
10	Conception de rosée Femmes Mode d'été Sexy C...	NULL	August	2020	1000	11.00

The status bar at the bottom indicates the query was executed successfully, showing the server name 'HEWAGE2003\SQLEXPRESS (15.0...)', the instance 'HEWAGE2003\hp (58)', the database 'Wish_Ecommerce_DW', and the execution time '00:00:00' for 10 rows.

Conclusion

This assignment provided comprehensive hands-on experience in designing and implementing a complete Data Warehousing and Business Intelligence solution using Microsoft SQL Server, SSIS, and industry-standard dimensional modelling techniques applied to a real-world e-commerce dataset.

The Wish Summer Products dataset proved to be an excellent choice for this exercise. As a genuine dataset capturing raw product listing and sales data from the Wish marketplace, it presented all the characteristics necessary to demonstrate the full spectrum of DW and BI concepts - including multiple source types, meaningful dimensions, aggregatable measures, and slowly changing attributes.

The ETL development in Task 5 provided deep practical exposure to SQL Server Integration Services. Building the two-package architecture - one for extracting from sources to staging and another for transforming and loading from staging to the data warehouse - reinforced the importance of the staging layer as a safe, auditable intermediate zone.

Task 6 introduced the concept of accumulating snapshot fact tables, which differ fundamentally from transactional fact tables in that rows are updated as a process moves through stages rather than being insert-only. Implementing a separate SSIS package to update the `accm_txn_complete_time` column and automatically recalculate `txn_process_time_hours` demonstrated how data warehouse fact tables can capture the lifecycle of a business process.

Task	Status	Key Deliverable
Task 1 - Dataset Selection	Complete	Wish Summer Products dataset with ER diagram
Task 2 - Data Sources	Complete	SQL Server DB + CSV flat file (2 source types)
Task 3 - Architecture	Complete	4-layer DW/BI architecture diagram
Task 4 - DW Design	Complete	Star schema: 3 dimensions + 1 fact table, SQL implemented
Task 5 - ETL Development	Complete	2 SSIS packages, SCD Type 1 & 2, 4,719 rows loaded
Task 6 - Accumulating Fact	Complete	Package 03 updates completion times and process hours